

```
type ConnectionPool struct {
    queue      chan *sql.DB
    maxSize    int
    currentSize int
    lock       sync.Mutex
    isNotFull  *sync.Cond
    isNotEmpty *sync.Cond
}
```

Here, the *ConnectionPool* struct contains the *queue*, *maxSize*, *currentSize*, *lock*, *isNotFull*, and *isNotEmpty* fields. The *queue* field is a channel that holds pointers to *sql.DB* connections. *sql.DB* type belongs to Go's built-in database/SQL package. The latter provides a generic interface around SQL or SQL-like databases. This interface is implemented by the github.com/mattn/go-sqlite3 package which we will be using as an SQLite3 driver.

The *maxSize* field represents the maximum number of connections that the pool can have, and the *currentSize* field represents the current number of connections in the pool. The *lock* field is a mutex ([https://en.wikipedia.org/wiki/Lock_\(computer_science\)](https://en.wikipedia.org/wiki/Lock_(computer_science))) that ensures concurrent access to shared memory is synchronised. The *isNotFull* and *isNotEmpty* fields are condition variables that allow for efficient waiting and are used to signal when the pool is not full and not empty, respectively.

The *sync.Cond* is a synchronisation primitive in Go that allows multiple goroutines to wait for a shared condition to be satisfied. It is often used in conjunction with a mutex, which provides exclusive access to a shared resource (in this case the queue), to coordinate the execution of multiple goroutines. Channels can also be used for synchronisation, but they come with some overhead in terms of memory usage and complexity. In this case, the use of *sync.Cond* provides a simpler and more lightweight alternative as it allows for efficient signalling for waiting goroutines.

By using *sync.Cond*, the implementation can ensure that goroutines waiting on the condition will be woken up only when the condition is actually met, rather than relying on a buffer channel that might have stale data. This improves the overall performance and reduces the likelihood of race conditions or deadlocks.

Getting a connection object from the pool

Next, we will implement a *Get* method which will return a database object from an existing *ConnectionPool*:

```
func (cp *ConnectionPool) Get() (*sql.DB, error) {
    cp.lock.Lock()
    defer cp.lock.Unlock()
```

```
    // If queue is empty, wait

    for cp.currentSize == 0 {
        fmt.Println("Waiting for connection to be added back
in the pool")
        cp.isNotEmpty.Wait()
    }

    fmt.Println("Got connection!! Releasing")
    db := <-cp.queue
    cp.currentSize--
    cp.isNotFull.Signal()

    err := db.Ping()
    if err != nil {
        return nil, err
    }

    return db, nil
}
```

This function, *Get()*, retrieves a connection from the pool. First, it acquires the lock to ensure exclusive access to the shared state of the connection pool. If the pool is currently empty, the function waits until a connection is added back to the pool.

Once a connection is available, the function dequeues it from the queue, decrements *currentSize*, and signals that the pool is not full. It then checks whether the connection is still valid by calling *Ping()*. If the connection is not valid, an error is returned, and the connection is not returned to the caller. If the connection is valid, it is returned to the caller.

Adding a connection object to the pool

Moving on, we introduce an *Add* method whose responsibility it will be to add the connection object to the pool once it has been used:

```
func (cp *ConnectionPool) Add(db *sql.DB) error {
    if db == nil {
        return errors.New("database not yet initiated. Please
create a new connection pool")
    }

    cp.lock.Lock()
    defer cp.lock.Unlock()

    for cp.currentSize == cp.maxSize {
        fmt.Println("Waiting for connection to be released")
        cp.isNotFull.Wait()
    }
}
```